National University of Computer & Emerging Sciences

Data Structures — Quiz: Array ADT and Its Implementation

Version A • Time: 15 minutes • Total marks: 10

Name: Model Solution	Roll No: _____	Section: _____

Answer in the space provided. Show your working — a correct final answer with no working earns half marks.

Q1. Error spotting and code output [4 marks]

(a) The following two lines are used for the array-of-arrays view of a 2-D array. The second line does not compile. State why, and rewrite the pair correctly. [2]

```
double  RowOfTable[4];
RowOfTable  score[10];
```

↳ RowOFTable is a variable and not a datatype, hence its use as a datatype in line 2 causes a compiler error as variables cannot declare other variables.

↳ Corrected

```
typedef double RowOfTable[4];
RowOfTable score[10];
```

OR

```
using RowOfTable = double[4];
```

(b) The two outputs below are different, even though both refer to the same array. State what each line prints and explain the cause in one sentence. [2]

```
void show(int a[]) {
    cout << sizeof(a) / sizeof(a[0]) << endl;   // line 1
}
int main() {
    int v[10];
    show(v);
    cout << sizeof(v) / sizeof(v[0]) << endl;   // line 2
}
```

↳ line 1 prints 2 (or 1) depending upon the architecture, while line 2 prints 10. (Considering size of int = 4 bytes)

↳ Cause: When an array is passed to a ftn, it decays into pointer to the first element of the array, hence sizeof(a) is actually sizeof(int*). In the main ftn, size of (v) is evaluated as 10 × 4 = 40 bytes

Q2. Address arithmetic [3 marks]

An array is declared as `int table[4][6];` The base address of `table` is **2000** and `sizeof(int)` = 4 bytes.

(a) How many bytes does the whole array occupy? [1]

4 × 6 × 4 = 96 bytes (ranging from address 2000 to 2095)

(4 × 6: no. of elements; 4: sizeof (int))

(b) Compute the address of `table[2][3]` under **row-major** storage. Show the formula and the number. **[1]**

base + (i × NUM-COLS + j) × sizeof(int)

= 2000 + (2 × 6 + 3) × 4

= 2000 + (15) × 4 = 2060

(c) Compute the address of the same element if the language stored the array **column-major** instead. **[1]**

base + (j × NUM-ROWS + i) × sizeof(int)

= 2000 + (3 × 4 + 2) × 4

= 2000 + (14) × 4

= 2056

Q3. Short answer — justify your reasoning **[3 marks]**

(a) For an array holding n elements, `a[i]` is O(1) but inserting a new element at position 0 is O(n). Give the reason for each, one sentence each. **[2]**

↳ Access is O(1) since all elements are stored contiguously and have the same size, so the address computation is done by: base + i × sizeof(T) which is independent of 'n'.

↳ Insertion at pos. 0 is O(n) since each element (for a total of 'n' elements) must be shifted to the right by 1 to maintain order and contiguity and make space for the new element.

(b) `std::vector` doubles its capacity when it runs out of room, rather than growing by one slot. Why does doubling matter? **[1]**

Growing by one slot copies $1+2+\dots+n \approx \frac{n^2}{2}$ elements over 'n' appends which is quadratic. Doubling the capacity copies $1+2+4+\dots+n < 2n$ elements over 'n' appends which is linear O(n), i.e. amortised O(1) per append. Hence, constant growth gives us an arithmetic series and is $O(n^2)$, while scaling (multiplying by a factor) is linear.